

Das Terminierungs-Orakel

Contents

Das Terminierungs-Orakel: Eine typisierte Konvergenz-Referenz für agentische Softwareentwicklung	1
Abstract (English)	1
Kurzfassung (Deutsch)	2
1. Problem	2
2. Methode	4
3. Fallstudie A — Runenruf: der Existenzbeweis	5
4. Fallstudie B — Ledger: das Gegenstück	6
5. Reflexive Selbstanwendung	7
6. Grenzen	9
7. Verwandte Arbeiten	10
8. Fazit	11
9. Reproduzierbarkeit	11
10. Literatur	12

Das Terminierungs-Orakel: Eine typisierte Konvergenz-Referenz für agentische Softwareentwicklung

Cong Chanh Vinzenz Nguyen — .NET-Architekt, Deutsche Börse AG

Abstract (English)

Agentic software development is increasingly framed as *Loop Engineering* (Boris Cherny, Anthropic: „my job is to write loops”). We argue that the open problem is not the loop but its **termination oracle**: a loop that halts on „the agent says done” or on process exit code 0 is bound to no external, checkable reference, and degenerates into brute force — token burn, a black box, accumulated cognitive and intent debt. We replace the oracle with *convergence against a typed specification*. The reference is a typed F# discriminated-union graph (SPOT), one git-versioned JSON per node under `.spot/`, with a four-valued convergence state (**Aligned**, **Pending**, **Diverged**, **Orphaned**). Generator and oracle are architecturally separated: the generator cannot write code (enforced by a tool allowlist); the oracle `SetzeSpecAligned` marks a node **Aligned** only on a real green `dotnet test` run — test-PASS, not process exit 0. Review thus shifts from reading diffs to verifying convergence against a human-set invariant. We provide three reproducible, CI-green artifacts (all public on GitHub): an existence proof (**runenruf**, a game, 46/46 tests, one FsCheck property over every seed and command sequence); a counterpart with real IO and a changing requirement (**ledger-casestudy**, double-entry

settlement ledger, 5/5 tests, int64-cent, a caught mis-spec — „Falsifiable, after 4 tests”); and three stacked verification layers on a single invariant (value conservation): types, an FsCheck property, and a sorry-free Lean 4 proof over every booking sequence whose model includes the coverage/rejection path of the F# code (Core + omega, no Mathlib; `#print axioms` names only `propext`, `Quot.sound`). The remainder is the *setting* of the invariant — not self-foundable within the system (a regress result, not an impossibility proof), and assigned to the human by a normative choice (intent is external to the generator and accountable), not by any claim that a machine could not. This work stands **with** Carsten Lucke’s skepticism of full automation, not against it: one screw turn further.

Kurzfassung (Deutsch)

Agentische Softwareentwicklung wird heute als *Loop Engineering* gerahmt. Wir zeigen: Das Problem ist nicht der Loop, sondern sein **Terminierungs-Orakel**. Ein Loop, der auf „der Agent sagt fertig” oder auf Prozess-Exit-Code 0 terminiert, ist an keine prüfbare externe Referenz gebunden und verfällt zu Brute Force — Token-Verbrauch, Blackbox, akkumulierte Cognitive und Intent Debt. Wir ersetzen das Orakel durch *Konvergenz gegen eine getypte Spezifikation*: einen git-versionierten, typisierten F#-DU-Graphen (SPOT) mit vier Konvergenzzuständen. Generator und Orakel sind architektonisch getrennt — der Generator kann per Werkzeug-Allowlist keinen Code schreiben, das Orakel akzeptiert nur einen echten grünen `dotnet test`-Lauf. Review wird damit Verifikation gegen eine vom Menschen gesetzte Invariante statt Diff-Lektüre. Drei reproduzierbare, CI-grüne Artefakte belegen die Kette: ein Existenzbeweis (**runenruf**), ein Gegenstück mit echtem IO und veränderlichem Requirement (**ledger-casestudy**, gefangener Mis-Spec) und drei Verifikationsschichten — Typen, FsCheck-Property, sorry-freier Lean-4-Beweis — auf einem Invarianten. Der Rest ist das *Setzen* der Invariante — im System nicht selbst-fundierbar (ein Regress-Resultat, kein Unmöglichkeitsbeweis), dem Menschen durch eine normative Wahl zugewiesen (Intent ist extern und verantwortungsbefahet), nicht durch die Behauptung, eine Maschine könne es nicht. Diese Arbeit steht **mit** Carsten Luckes Skepsis gegenüber Voll-Automation — eine Schraube weiter.

1. Problem

1.1 Der Loop und sein blinder Fleck

Boris Cherny (Anthropic, Claude Code) fasst die gegenwärtige Praxis agentischer Softwareentwicklung in einem Satz zusammen: „my job is to write loops” — „Loop Engineering” (The New Stack, „Loop Engineering”, 2026). Ein Generator-Modell erzeugt einen Vorschlag, ein Schritt prüft ihn, das Ergebnis fließt zurück, der Loop wiederholt sich bis zu einem Abbruchkriterium. Die Architektur ist korrekt. Die offene Frage ist nicht die Schleife, sondern ihr **Terminierungs-Orakel**: woran erkennt der Loop, dass er fertig ist?

Wir behaupten: das Problem von Loop Engineering ist nicht der Loop, sondern das Terminierungs-Orakel. Terminiert ein Loop auf „der Agent sagt fertig” oder auf „der Prozess endet mit Exit-Code 0”, dann ist die Abbruchbedingung nicht an eine externe, prüfbare Referenz gebunden. Was bleibt, ist Brute Force: Token-Verbrauch ohne verbürgten Endzustand, eine Blackbox, deren Korrektheit der Mensch nachträglich durch Diff-Lektüre validieren muss.

1.2 Cognitive Debt und Intent Debt

Margaret-Anne Storey benennt die strukturelle Folge. In „From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI” (arXiv:2603.22106, 2026) definiert sie

Intent Debt als „the absence of externalized rationale that developers and AI agents need to work safely with code“. Wenn ein Agent Code erzeugt, dessen Begründung — die Absicht, gegen die er geprüft werden müsste — nirgends externalisiert vorliegt, dann akkumuliert das System Schuld, die kein Test sichtbar macht. Diff-Lektüre tilgt diese Schuld nicht; sie verschiebt sie auf den Reviewer, der die Absicht im Kopf rekonstruieren muss.

1.3 Ehrlicher Stand der Technik

Drei Befunde grenzen das Feld ehrlich ab.

Wachsende, aber begrenzte Reichweite (METR). „Measuring AI Ability to Complete Long Software Tasks“ (arXiv:2503.14499, 2025) misst die Fähigkeit von Modellen über die Metrik „50%-task-completion time horizon“ — die Aufgabendauer, bei der ein Modell mit 50 % Wahrscheinlichkeit erfolgreich ist. Spitzenmodelle Anfang 2025 liegen bei rund 50–110 Minuten, mit einer Verdopplung etwa alle 7 Monate. Das ist ein realer, gemessener Trend — und zugleich eine harte Aussage über die heutige Grenze: jenseits dieses Horizonts sinkt die Erfolgsrate. Ein Loop, der über diese Grenze hinaus autonom laufen soll, braucht ein Orakel, das ihn anhält, bevor er driftet.

Unentscheidbarkeit als Motivation, nicht als Beweis (Rice). Rice (1953) zeigt: nicht-triviale semantische Eigenschaften von Programmen sind unentscheidbar. Wir verwenden dies ausschließlich als strukturelle Motivation, nicht als Beweis. Der Satz trifft Mensch **und** Maschine symmetrisch. Daraus folgt **nicht** „der Mensch kann, was die Maschine nicht kann“ — diese Schlussfigur (Penrose/Lucas) lehnen wir ab. Was folgt, ist enger und schärfer: die gewünschte semantische Eigenschaft kann nicht aus dem Programm allein entschieden werden; sie muss **gesetzt** werden. Jemand wählt die Invariante. Diese Setzung liegt außerhalb des Entscheidungsproblems.

Verifizierer-akzeptiert $\neq$ korrekt (VeriAct). Md Rakib Hossain Misu, Iris Ma und Cristina V. Lopes zeigen in „VeriAct: Beyond Verifiability — Agentic Synthesis of Correct and Complete Formal Specifications“ (arXiv:2604.00280, 2026): viele Spezifikationen, die ein Verifizierer akzeptiert, sind dennoch inkorrekt oder unvollständig. Das ist die entscheidende Einschränkung gegen jede naive Loop-Automatisierung — auch gegen unsere eigene: das Orakel allein genügt nicht. Ein grüner Verifizierer beweist nicht, dass die **richtige** Eigenschaft geprüft wurde. Der Mensch, der die Invariante setzt, ist nicht wegoptimierbar.

Die externe Verifikation als Quelle der Garantie ist bei Subbarao Kambhampati u. a. präzise gefasst: „LLMs Can’t Plan, But Can Help Planning in LLM-Modulo Frameworks“ (arXiv:2402.01817, 2024) — die Garantie kommt vom externen Verifizierer, nicht vom Generator. Property-orientierte Orakel für LLM-erzeugten Code sind in „Effective LLM Code Refinement via Property-Oriented and Structurally Minimal Feedback“ (PGS, arXiv:2506.18315, 2025) untersucht.

1.4 Die Leitfrage

Shuvendu K. Lahiri benennt den intellektuellen Kern als akademische Grand Challenge: „Intent Formalization: A Grand Challenge for Reliable Coding in the Age of AI Agents“ (arXiv:2603.17150, 2026). Der Mensch steht am Spec-Gate; Review ist Verifikation gegen eine Referenz, nicht Lektüre eines Diffs; „trifft die Spec die Welt“ ist der Rest, der außerhalb des Prüfungsvorgangs liegt. Wir beanspruchen nicht, diese Idee erdacht zu haben — sie ist publiziert und benannt. Wir beanspruchen auch keine Priorität: uns ist keine publizierte Implementierung mit genau dieser Kombination bekannt — durchgängig getypte, lauffähige Kette mit architektonisch getrenntem Generator und Orakel und einem property- bzw. beweis-verifizierten Gate. Das ist eine Literatur-Aussage, kein Beweis.

Daraus folgt die Leitfrage dieses Papiers:

Wo liegt die Grenze, an der Verifikation auf eine gesetzte Invariante angewiesen ist — und lässt sie sich **typisieren**, das heißt als externalisierte, maschinenlesbare Referenz an genau die Stelle legen, an der jemand die Invariante setzen muss und wir den Menschen wählen?

Diese Arbeit steht **mit** Carsten Luckes Skepsis gegenüber Voll-Automation, nicht gegen sie — eine Schraube weiter. Wir teilen die Skepsis und liefern ihre konstruktive, falsifizierbare Konsequenz.

2. Methode

2.1 Konvergenz gegen Spec statt „Agent sagt fertig“

Wir ersetzen das Terminierungs-Orakel. Der Loop terminiert nicht, wenn der Generator behauptet, fertig zu sein, sondern wenn das System **gegen die Spec konvergiert**. Konvergenz ist ein typisierter, ablesbarer Zustand — nicht eine Einschätzung des Generators.

2.2 Das typisierte SPOT-Modell

SPOT (Single Point of Truth) ist ein typisierter F#-Discriminated-Union-Graph. Jeder Knoten liegt als ein git-versioniertes JSON unter `.spot/` — die Referenz ist damit externalisiert, versioniert und Teil der Historie, nicht ein flüchtiger Prompt-Kontext. Genau das ist das von Storey geforderte „externalized rationale“: die getypte Spec **ist** das externalisierte Rationale, das Intent Debt tilgt, statt sie auf den Reviewer zu verschieben.

Jeder Knoten trägt einen Konvergenzzustand aus vier Werten: **Aligned**, **Pending**, **Diverged**, **Orphaned**. Das aktuelle Selbst-Modell des Werkzeugs (Koschnag/cog-driven-development, v0.4.0) hat **66 Knoten**, **62 Aligned**, **4 Pending**, bei **36/36 grünen Unit-Tests**. Die vier Pending-Knoten sind ehrlich offen — zwei davon sind LLM-Ergebnis-Kriterien, die **nicht** mechanisch prüfbar sind und daher bewusst nicht auf **Aligned** gesetzt wurden. Im Zuge ehrlicher Konvergenz wurden 5 abgelöste Knoten gelöscht und 3 end-to-end verifizierte UI-Specs promotet. Die reflexive Selbstanwendung dieses Modells führen wir in Abschnitt 5 aus.

2.3 Generator/Orakel-Trennung

Der entscheidende architektonische Schnitt ist die Trennung von **Generator** und **Orakel**.

- Der **Generator** ist die restringierte Engine. Über eine Werkzeug-Allowlist hat sie **kein Schreibrecht auf den Code-/Gate-Pfad** — nur Modell-Werkzeuge (sie schreibt Spec-Knoten, keinen Code). Die Trennung ist strukturell erzwungen, nicht durch Konvention.
- Das **Orakel** ist die Funktion `SetzeSpecAligned`. Ein Spec-Knoten wird **nur** dann **Aligned**, wenn ein echter `dotnet test`-Lauf grün ist — Test-PASS, nicht Prozess-Exit-0. Die Unterscheidung ist tragend: ein Prozess kann mit 0 enden, ohne dass ein einziger Test bestanden wurde.

Diese Trennung adressiert VeriAct direkt: das Orakel akzeptiert nur einen echten grünen Test, aber **welche** Invariante geprüft wird, setzt der Mensch — der Verifizierer wird nicht zur alleinigen Korrektheitsquelle gemacht.

Reflexiv abgesichert ist die Trennung durch den Selbst-Gate-Knoten `spec-gate-selbst-hart`: ein Test, in dem das System eine Invariante über sein **eigenes** `.spot/-Modell` prüft — kein Knoten darf

Aligned und vom Typ **Test** sein ohne echten Marker. Das Gate hat Zähne gegen sich selbst; die volle Behandlung folgt in Abschnitt 5.

2.4 Verschiebung des Akzeptanzkriteriums

Damit verschiebt sich, was Review bedeutet: von **Diff-Lektüre** zu **Konvergenz gegen eine getypte Referenz**. Der Mensch liest nicht den erzeugten Code-Diff Zeile für Zeile; er prüft, ob das Modell gegen die von ihm gesetzte Spec konvergiert. Die empirische Stütze dafür liefern die beiden Fallstudien in den Abschnitten 3 und 4 sowie die gemessene Kompression in Abschnitt 4.2.

2.5 Abgrenzung zu Kiro / GitHub Spec Kit

Spec-getriebene Werkzeuge wie Kiro oder das GitHub Spec Kit lassen **dasselbe** System Spec **und** Code **und** Test erzeugen. Dann ist die Referenz nicht unabhängig vom Prüfling: der Generator schreibt seine eigene Prüfung. Hier ist die Referenz unabhängig — der Generator kann per Allowlist keinen Code schreiben, das Orakel verlangt einen echten grünen Test gegen eine separat gesetzte Invariante, und der reflexive Selbst-Gate-Knoten prüft, dass diese Trennung nicht unterlaufen wird. Abschnitt 7 vertieft die Abgrenzung im Kontext der verwandten Arbeiten.

3. Fallstudie A — Runenruf: der Existenzbeweis

Runenruf ist ein Spiel, und im Kontext dieses Whitepapers der schlichteste mögliche Beleg: der Nachweis, dass die Kette — getypte Spec, getrennter Generator, Orakel gegen echten grünen Test — überhaupt auf einer geschlossenen Domäne durchläuft. Wir nennen ihn den Existenzbeweis und nicht mehr.

Aufbau. Die Domäne lebt in `src/Runenruf.Domain/Sim.fs` und umfasst rund 125 Zeilen Code (ohne Leer- und Kommentarzeilen). Die Simulation ist deterministisch — gleicher Seed ergibt bit-gleichen Zustands-Hash. Das ist keine Bequemlichkeit, sondern die Voraussetzung dafür, dass ein Orakel überhaupt entscheiden kann: ein nicht-deterministischer Lauf hätte kein reproduzierbares Urteil.

Testlage. `dotnet test tests/Runenruf.Tests` liefert 46/46 grün. Wir haben die Zahl gegen die Quelle gezählt: 46 mit [`<Fact>`] annotierte Tests in `tests/Runenruf.Tests/Tests.fs`. Das Orakel des CDD-Modells setzt einen Spec-Knoten nur dann auf **Aligned**, wenn ein echter `dotnet test`-Lauf grün ist — Test-PASS, nicht Prozess-Exit-0. Diese Unterscheidung ist tragend: ein Prozess kann mit Exit-Code 0 enden, ohne dass je eine Zusicherung geprüft wurde.

Die Property. Der Kern der Fallstudie ist ein einzelner Knoten: `spec-siegel-lager-nichtnegativ`. Zwei beispielbasierte Tests (`-test-1`, `-test-2`) decken konkrete Befehlsfolgen ab. Daneben steht eine `FsCheck-Property` (`-property`), die denselben Invarianten — an jedem Tick bleibt jeder Lagerstand ≥ 0 — auf „für jeden Seed und jede Befehlsfolge“ hebt. Der Quelltext formuliert das wörtlich: „für jeden Seed und jede Befehlsfolge bleibt an jedem Tick jeder Lagerstand ≥ 0 “. Das ist der Unterschied zwischen einer Beispieltabelle und einer Aussage: die Tabelle prüft, was wir uns ausgedacht haben; der Generator prüft, was wir uns nicht ausgedacht haben.

Ehrlichkeit. Runenruf ist die freundlichste denkbare Domäne. Sie ist geschlossen und deterministisch. Die Frage, an der jede ernsthafte Spezifikation hängt — „trifft die Spec die Welt“ — ist hier fast leer, weil die Spec die Welt *ist*. Es gibt keine externe Realität, gegen die der formalisierte Wille auseinanderlaufen könnte; das Spiel hat keinen Referenten außerhalb seines eigenen Regelwerks.

Runenruf belegt also, dass die Mechanik trägt. Es belegt nicht, dass sie unter dem schwierigeren Teil des Problems trägt. Dafür ist die zweite Fallstudie da.

4. Fallstudie B — Ledger: das Gegenstück

Ledger (Koschnag/ledger-casestudy) ist ein doppisches Hauptbuch für Settlement, gebaut, um genau die Härte zu liefern, die Runenruf fehlt: echtes IO, ein sich änderndes Requirement und eine Spezifikation, deren Treffen mit der Welt nicht trivial ist.

Aufbau. F# auf .NET 9. Beträge sind `int64` (Cent), kein Float — der Quelltext begründet es selbst: „Werte sind `int64` (Cent) — kein Float, damit Erhaltung exakt gilt.“ Die Domäne in `src/Ledger/Ledger.fs` umfasst rund 41 Zeilen Code (ohne Leer- und Kommentarzeilen). `dotnet test` liefert 5/5 grün: vier Property-Tests und ein Beispieltest.

Echtes IO. Anders als bei Runenruf gibt es Persistenz. Der Knoten `spec-replay-treue` prüft, dass ein auf Platte geschriebenes und neu geladenes Journal denselben Zustand rekonstruiert wie der direkte Lauf — `Journal.replay anfang pfad = direkt`. Das Orakel urteilt hier über tatsächliche Plattenschreibvorgänge, nicht über reine Funktionen.

Das veränderliche Requirement. Gebühren kamen *nach* der ersten Spezifikation. Das ist der realistische Fall: die Anforderung war nicht falsch, sie war unvollständig, und sie änderte sich. Der naive erste Entwurf zog die Gebühr beim Sender ab und schrieb sie nirgends gut. Wert verschwand.

Der gefangene Mis-Spec. Die Invariante `spec-werterhaltung` — die Gesamtsumme aller Konten ist über jede Buchung konstant — fing genau diesen Fehler. Beim Lauf gegen den naiven Entwurf meldet das Orakel wörtlich:

`Falsifiable, after 4 tests`

`FsCheck` brauchte vier generierte Folgen, um den Verlust zu provozieren. Niemand musste den Diff lesen, um den Fehler zu sehen; die Eigenschaft fiel. Der Fix ist Doppik: die Gebühr wird als Gegenbuchung geführt, nicht als Verlust. Wert wird verschoben, nicht vernichtet. Das reproduzierbare Skript `demo-gate-at-failure.sh` führt die ganze Bewegung vor — korrekt grün, naives Modell injiziert (Gebühr verschwindet), `spec-werterhaltung` wird rot (`Falsifiable`), `revert`, wieder grün.

Das Gate beim Scheitern. Dies ist das stärkste Bild dieses Whitepapers. Nicht der grüne Lauf, sondern der rote. Ein Loop ohne externes Orakel hätte den naiven Entwurf produziert, mit Exit-Code 0 quittiert und Wert lautlos vernichtet — die Eigenschaft, die ihn entlarvt, existierte nicht. Hier existiert sie, sie ist unabhängig vom Prüfling formuliert, und sie fällt. Review ist an dieser Stelle nicht die Lektüre des Diffs, sondern die Konvergenz gegen die gesetzte Invariante.

4.1 Drei Verifikationsschichten an einem Invarianten

Derselbe Invariant — Werterhaltung — trägt in diesem Repository drei voneinander getrennte Garantien, von schwach und total nach stark und allquantifiziert:

1. **Typen.** `int64`-Cent schließt illegale Float-Rundung konstruktiv aus. Eine ganze Klasse von Erhaltungsverletzungen kann syntaktisch gar nicht erst entstehen. Diese Schicht gilt für jeden Wert, prüft aber nur das Repräsentationsmittel, nicht die Buchungslogik.
2. **FsCheck-Property.** Über endlich viele generierte Zufallsfolgen *geprüft*. Diese Schicht fand den Mis-Spec (`Falsifiable, after 4 tests`). Sie ist falsifizierend und stark in der Praxis,

aber sie ist Stichprobe, kein Beweis: sie deckt die Folgen ab, die der Generator erzeugt, nicht alle.

3. **Lean-4-Beweis.** `proofs/Werterhaltung.lean` *beweist* die Erhaltung für *jede* Buchungsfolge. Das Theorem `werterhaltung` quantifiziert über `List Buchung` — also über alle endlichen Folgen, nicht über eine Stichprobe. Der Beweis kommt ohne Mathlib aus (Core plus `omega`) und ist sorry-frei. `#print axioms Ledger.werterhaltung` steht im Quelltext und bezeugt, dass nur Standard-Axiome eingehen (`propext`, `Quot.sound`) und kein `sorryAx`. Der CI-Job `lean-proof` ist auf GitHub-Hardware grün.

Die drei Schichten sind nicht redundant, sie sind gestaffelt: die Typen verhindern eine Fehlerklasse vorab, die Property fängt Fehler im laufenden Entwurf gegen ein änderndes Requirement, der Beweis schließt den Eingaberaum vollständig. Ein sorry-freier Beweis ohne Mathlib mit ausgewiesener Axiomliste ist die stärkste Garantie, die wir hier geben — sie gilt allquantifiziert, nicht stichprobenweise.

4.2 Kompression — gemessen, konditional

Eine 1-Zeilen-Invariante deckt einen unbeschränkten Eingaberaum ab: als FsCheck-Generator über beliebige Folgen, als Lean-Theorem über `List Buchung` sogar allquantifiziert bewiesen. Dem stehen rund 41 Zeilen Ledger-Domäne und rund 125 Zeilen Runenruf-Simulation gegenüber. Eine Beispieltabelle, die dieselbe Garantie liefern wollte, ginge gegen unendlich viele Zeilen — für die bewiesene Schicht ist sie prinzipiell nicht erreichbar.

Operativ heißt das: wir reviewen eine Zeile, keinen Diff von rund 41 bis 125 Zeilen. Der Einwand „die Spec ist nur Code im Trenchcoat“ — die Behauptung, eine Spezifikation sei bloß eine umbenannte Reimplementierung — ist in *diesen* Fällen widerlegt: der Invariant ist echt kürzer als seine Implementierung und in der Lean-Schicht von ihr logisch unabhängig. Das ist eine konditionale Aussage über zwei Fallstudien, keine universelle Äquivalenzbehauptung. Wir behaupten nicht, dass jede Spec gegenüber ihrem Code komprimiert; wir zeigen, dass diese beiden es nachweislich tun.

5. Reflexive Selbstanwendung

Der härteste Test einer Methode ist, ob sie sich selbst überlebt. Die CDD-IDE wird mit denselben Mitteln entwickelt, die sie propagiert: Ihr eigenes Selbst-Modell liegt git-versioniert unter `.spot/` als typisierter `F#-DU-Graph`, ein JSON pro Knoten. Wir wenden das Konvergenz-Orakel also nicht nur auf fremde Domänen an, sondern auf den Erzeuger selbst.

5.1 Der reflexive Gate-Knoten

Es gibt einen Spec-Knoten `spec-gate-selbst-hart` mit der Intention: Ein Test-Knoten gilt nur dann als `Aligned`, wenn ein echter Test-Marker im Testcode existiert — nicht durch bloße Behauptung. Das zugehörige Kriterium (Given/When/Then) ist als ausführbarer Test `spec-gate-selbst-hart-test-1` realisiert. Dieser Test lädt zur Laufzeit das eigene `.spot/-Modell`, scannt die Testprojekte nach `[<Trait("spot", id)>]`-Markern und prüft die Invariante:

Jeder als `Aligned` markierte Test-Knoten im Selbst-Modell besitzt einen echten Trait-Marker im Testcode — es gibt kein `Aligned` ohne Test.

Verwaiste Knoten — `Aligned`, aber ohne Marker — lassen den Test fehlschlagen. Zusammen mit grüner Suite in CI folgt daraus konditional: Wenn dieser Test grün ist, dann existiert zu jedem

Aligned-Test-Knoten ein realer, grüner Test. Das System gattet seinen eigenen Drift in derselben Suite, die es validiert.

Dies schließt exakt die Defektklasse, die ein naives Orakel hätte: einen Test-Knoten **Aligned** zu setzen, weil ein Marker präsent ist, statt weil die Suite grün läuft. Das Orakel **SetzeSpecAligned** setzt einen Spec-Knoten nur dann **Aligned**, wenn ein echter **dotnet test**-Lauf grün ist (Test-PASS, nicht Prozess-Exit-0) — und diese Disziplin wird hier auf das Selbst-Modell zurückgebogen.

5.2 Ehrliche Selbst-Konvergenz

Das Selbst-Modell umfasst aktuell 66 Knoten: 62 **Aligned**, 4 **Pending**. Die Test-Suite zählt 36 von 36 grünen Tests. Diese Zahlen sind das Resultat einer dokumentierten, ehrlichen Konvergenz, nicht eines geschönten Endzustands:

- **5 abgelöste Knoten gelöscht** — Spezifikationen, die von neueren abgelöst wurden, bleiben nicht als Karteileichen **Aligned**, sondern verschwinden.
- **3 e2e-verifizierte UI-Specs promotet** — von **Pending** nach **Aligned** erst nach echter End-to-End-Verifikation.
- **4 Knoten ehrlich Pending** — sie werden nicht vorzeitig grüngefärbt.

Bemerkenswert an den 4 verbliebenen **Pending**-Knoten: Der Gate-Spec-Knoten **spec-gate-selbst-hart** selbst ist **Pending**, obwohl sein Test **spec-gate-selbst-hart-test-1** **Aligned** und grün ist. Das ist kein Widerspruch, sondern die geforderte Ehrlichkeit: Der Test prüft eine notwendige Teilbedingung; das volle Kriterium des Spec-Knotens (die Invariante über das gesamte Modell unter allen künftigen Mutationen) ist damit nicht abschließend mechanisch eingelöst. Zwei der vier **Pending**-Knoten tragen LLM-Ergebnis-Kriterien, die nicht mechanisch prüfbar sind — sie bleiben **Pending**, weil kein verifizierbares Orakel existiert, das sie ehrlich auf **Aligned** heben könnte. Wir setzen **Aligned** nicht, wo wir es nicht beweisen können.

5.3 Die reflexive Grenze

Hier liegt der entscheidende Punkt, und wir formulieren ihn ohne Übertreibung. Das System gattet seinen Code. Es kann sein Gate nicht begründen.

Die Invariante „kein **Aligned** ohne echten Test“ ist eine Setzung. Sie steht nicht im Entscheidungsverfahren des Systems; sie wurde von außen — vom Menschen am Modell-Gate — gewählt. Das System kann prüfen, ob seine Knoten dieser Invariante genügen; es kann nicht aus sich heraus entscheiden, ob diese Invariante die richtige ist. Die Meta-Invariante ist die Setzung des Menschen.

Strukturell — und ausdrücklich nur als Motivation, nicht als Beweis — spiegelt das die Lage metamathematischer Limitationsresultate: Ein hinreichend ausdrucksstarkes System kann seine eigene Korrektheitsnorm nicht aus sich selbst heraus vollständig fundieren (Gödel, Tarski). Wir wenden diese Theoreme hier nicht an — ihre Voraussetzungen (Repräsentierbarkeit, Konsistenz, rekursive Axiomatisierung) prüfen wir nicht, und unser **.spot/-Graph** ist keine arithmetische Theorie. Es ist eine strukturelle Analogie: Wer die Invariante setzt, steht außerhalb des Prüfungsvorgangs. Präzise getrennt heißt das zweierlei. (i) **Strukturell (Regress)**: Die Grund-Invariante ist im System nicht selbstfundierbar — jede Begründung bräuchte eine weitere Invariante, ein Regress, der nur durch einen Anker *außerhalb* des Prüfungsvorgangs terminiert. Das ist ein Argument, kein in diesem Text geführter formaler Beweis. (ii) **Normativ (kein Berechenbarkeits-Resultat)**: Dass *der Mensch* diesen Anker setzt, ist eine Wahl, keine Unmöglichkeitssaussage über Maschinen. Wir behaupten ausdrücklich *nicht*, eine Maschine könne keine Invariante setzen — das wäre der Penrose/Lucas-Fehlschluss,

den wir ablehnen (Rice (1953) trifft Mensch und Maschine symmetrisch). Wir *wählen* den Menschen, weil Intent extern zum Generator und verantwortungsbehaftet ist. Ein zweiter Agent könnte die Invariante setzen — dann ist der Mensch verschoben, nicht durch ein Theorem als notwendig erwiesen; genau diese Ehrlichkeit verlangt unser eigener Anti-Penrose-Standard.

Der reflexive Befund ist damit präzise und bescheiden: Das System härtet sein eigenes Gate gegen Drift — und macht zugleich sichtbar, dass der äußerste Bezugspunkt eine menschliche Setzung bleibt.

6. Grenzen

Wir benennen die Grenzen ungeschönt, weil ein Claim-Do-Gap die einzige unverzeihliche Sünde wäre.

- **Zwei kleine Fallstudien.** Der Existenzbeweis ruht auf `runenruf` (Spiel, 46/46 Tests, Domäne ~125 LoC) und `ledger-casestudy` (doppisches Hauptbuch, 5/5 Tests, Domäne ~41 LoC). Das ist kein großmaßstäbliches, produktives System. Es belegt, dass die Kette lauffähig und reproduzierbar existiert — nicht, dass sie auf beliebige Größenordnungen skaliert.
- **Die freundlichste Domäne ist die schwächste Evidenz.** `runenruf` ist geschlossen, deterministisch, headless simulierbar. Dort ist „Trifft die Spec die Welt?“ fast leer, weil die Spec die Welt *ist*. Das macht es zum sauberen Existenzbeweis der Mechanik, aber zur schwächsten Evidenz für den eigentlich harten Rest. `ledger-casestudy` ist das Gegenstück mit echtem IO (Journal auf Platte + Replay) und einem veränderlichen Requirement — dort fängt die Invariante eine reale Mis-Spec. Beides zusammen ist zwei Datenpunkte, keine Statistik.
- **Lean nur für einen Invarianten.** Der Werterhaltungs-Satz ist in `proofs/Werterhaltung.lean` für jede Buchungsfolge bewiesen, sorry-frei (`#print axioms` nennt nur `propxt`, `Quot.sound`; kein `sorryAx`), ohne Mathlib (Core + `omega`), CI-Job grün; das Lean-Modell enthält den Deckungs-/Ablehnungspfad und ist damit dasselbe Objekt wie der F#-Code. Das ist genau *ein* Invariant in genau einer Fallstudie — die übrigen Eigenschaften (Nichtnegativität, Isolation, Replay-Treue) sind bisher nur FsCheck-geprüft, nicht Lean-bewiesen; die Drei-Schichten-Kette ist also derzeit nur für die Werterhaltung geschlossen. Weitere Beweise sind Roadmap, nicht erbracht.
- **Kein Issue-Tracker, keine unbeschränkte Autonomie.** Es gibt keine offene Agenten-Schleife, die ohne menschliches Gate Tickets abarbeitet. Der Generator kann per Werkzeug-Allowlist keinen Code schreiben; das Orakel ist ein realer Testlauf. Das ist Absicht — aber es bedeutet, dass wir keine Aussage über autonomen Dauerbetrieb machen.
- **Der Mensch am Modell-Gate bleibt.** Das ist eine operationale, normative Grenze, keine metaphysische Behauptung über Maschinenfähigkeit. Wir behaupten nicht, ein Mensch könne prinzipiell etwas, das eine Maschine nie könne. Wir behaupten: Das *Setzen* der Grund-Invariante ist im System nicht selbst-fundierbar (Regress, kein Unmöglichkeitsbeweis), und dass es in der vorliegenden Architektur der Mensch tut, ist eine Wahl — Intent ist extern zum Generator und verantwortungsbehaftet. Zwei **Pending-Knoten** mit LLM-Ergebnis-Kriterien zeigen die Stelle, an der kein mechanisches Orakel verfügbar ist.
- **„Verifizierer-akzeptiert $\neq$ korrekt“.** Misu, Ma und Lopes (VeriAct, arXiv:2604.00280) zeigen, dass viele verifizierer-akzeptierte Spezifikationen inkorrekt oder unvollständig sind. Das trifft uns mit: Ein grünes Orakel beweist Konvergenz von Code gegen Spec, nicht die Korrektheit der Spec gegen die Welt. Genau dieser Rest ist die menschliche Setzung — und er ist nicht wegautomatisiert.

7. Verwandte Arbeiten

Wir grenzen den Beitrag exakt ab.

Lahiri, „Intent Formalization: A Grand Challenge for Reliable Coding in the Age of AI Agents” (arXiv:2603.17150, 2026). Der intellektuelle Kern — der Mensch am Spec-Gate, Verifikation statt Diff-Lektüre, „Trifft die Spec die Welt?” als der Rest, der außerhalb des Prüfvorgangs liegt — ist hier als benannte akademische Grand Challenge publiziert. Cong ist also nicht der Denker der Idee. Wir beanspruchen *keine* Priorität — „erster” wäre eine unbeweisbare Negativ-Existenzaussage über das ganze Feld. Wir beanspruchen konditional: Uns ist keine publizierte Implementierung mit genau dieser Kombination bekannt — getypte externalisierte Spec + architektonisch getrennter Generator/Orakel (Werkzeug-Allowlist ohne Generator-Schreibrecht) + property-/beweis-verifiziertes Gate. Neuheit ist eine Literatur-Aussage, kein Beweis; sie steht und fällt mit der nächsten Nachbarschaft (VeriAct).

Storey, „From Technical Debt to Cognitive and Intent Debt” (arXiv:2603.22106, 2026). Definiert Intent Debt als „the absence of externalized rationale that developers and AI agents need to work safely with code”. Die getypte Spec unter `.spot/` ist dieses externalisierte Rationale: git-versioniert, typisiert, gegen den Code konvergenz-gemessen. Der Beitrag ist eine konkrete Form, Intent Debt zu zahlen statt nur zu benennen.

Kambhampati u. a., „LLMs Can’t Plan, But Can Help Planning in LLM-Modulo Frameworks” (arXiv:2402.01817, 2024). Stützt die Architekturentscheidung: Die Garantie kommt vom externen Verifizierer, nicht vom Generator. Unser Orakel (realer grüner Test) ist genau dieser externe Verifizierer; der LLM-Generator ist die Vorschlagsquelle, nicht die Garantiequelle.

Misu, Ma, Lopes, VeriAct (arXiv:2604.00280, 2026). Die nächste verwandte Arbeit zur getypten Spec-Synthese. Befund „verifizierer-akzeptiert $\neq$ korrekt” stützt unsere ehrliche Grenze: Das Orakel allein genügt nicht; der Mensch setzt die Invariante. Wir halten unsere Uniqueness-Behauptung gerade wegen dieser Nähe eng — VeriAct synthetisiert Spezifikationen agentisch; unser Beitrag ist die getrennte, reproduzierbare Generator/Orakel-Kette mit property- und beweis-verifiziertem Gate, nicht die Spec-Synthese selbst.

PGS, „Effective LLM Code Refinement via Property-Oriented and Structurally Minimal Feedback” (arXiv:2506.18315, 2025). Property-orientiertes Orakel für LLM-Code — methodisch verwandt zu unserer FsCheck-Property als einer der drei Verifikationsschichten auf dem Werterhaltungs-Invarianten.

METR, „Measuring AI Ability to Complete Long Software Tasks” (arXiv:2503.14499, 2025). Liefert die gemessene Grenze der heutigen Reichweite (50%-task-completion time horizon, Anfang 2025 ~50–110 Minuten, Verdopplung ~7 Monate) und motiviert damit die Notwendigkeit eines anhaltenden Orakels.

Kiro / GitHub Spec Kit. Die schärfste Abgrenzung. Dort erzeugt dasselbe System Spec *und* Code *und* Test — die Referenz ist nicht unabhängig vom Prüfling. In CDD ist sie es — aber zweiachsig, und wir sind genau: Generator $\perp$ Prüf-Orakel ist *mechanisch* erzwungen (Werkzeug-Allowlist ohne Schreibrecht, fälschungssicher — diese Achse ist real). Generator $\perp$ Spec-Autor gilt *nur*, wenn die Spec aus separater Quelle stammt; erzeugt dieselbe Modell-Instanz Spec *und* Code, korrelieren die Fehler (common-mode — genau der VeriAct-Fall), und der Mensch am Modell-Gate ist der Dekorrelator. Der ledger-Fund „Falsifiable, after 4 tests” wurde entsprechend gefangen, weil ein *Mensch* die stärkere Property und das geänderte Requirement als Gegen-Orakel einbrachte, nicht der Loop aus sich selbst. Review ist damit Konvergenz gegen die externalisierte Spec, nicht das Lesen

des Diffs — und das Prüf-Orakel prüft nicht sich selbst.

Das breitere Feld. Spezifikationsgetriebene Entwicklung ist 2026 die dominante Antwort auf „Vibe-Coding“ — neben Kiro und Spec Kit etwa Cursor, OpenSpec, BMAD, Google Antigravity und die kommerzielle Plattform Tessel (Spec als primäres Artefakt + Tests als Leitplanken; ihr Problem-Framing — Agenten, die „fertig“ melden, ohne es zu sein — deckt sich mit unserer Diagnose). Parallel ist „LLM + formale Verifikation“ ein eigenes Subfeld, *vericoding*: aus formaler Spec verifizierten Code synthetisieren und mit Lean/Dafny/Verus prüfen — etwa CLEVER (Benchmark für Lean-verifizierte Codegenerierung, arXiv:2505.13938) oder VeriGuard (verifizierte Codegenerierung mit getrenntem Generator/Prüfer, arXiv:2510.05156). Unser Beitrag ist damit weder „Spec als Wahrheit“ (Commodity) noch „Code gegen formale Spec verifizieren“ (vericoding), sondern deren Integration zu einer laufenden, getypten Methode — eine Architektur- und Integrationsaussage, kein Ideen-Primat.

Die Rahmung gegenüber dem Loop-Engineering-Diskurs (Cherny, „my job is to write loops“, The New Stack 2026) ist konstruktiv: mit Lucke, eine Schraube weiter. Das Problem ist nicht der Loop, sondern das Terminierungs-Orakel. Wir teilen die Skepsis gegen Voll-Automation; der Beitrag ist die falsifizierbare Konsequenz daraus.

8. Fazit

Carsten Lucke hat die Diagnose. Seine Skepsis gegenüber der Voll-Automation agentischer Softwareentwicklung ist berechtigt, und wir teilen sie: ein Loop, der auf „der Agent sagt fertig“ oder auf Prozess-Exit-Code 0 terminiert, ist Brute Force mit grünem Anstrich. Was wir hinzufügen, ist die Schraube danach — nicht eine Widerlegung der Diagnose, sondern ihre konstruktive, falsifizierbare Konsequenz.

Die Konsequenz ist präzise. Das Problem von Loop Engineering ist nicht der Loop, sondern sein Terminierungs-Orakel. Tauscht man das Orakel „Agent sagt fertig“ gegen „Konvergenz gegen eine externalisierte, getypte Spec“, architektonisch durchgesetzt durch einen Generator, der keinen Code schreiben kann, und ein Orakel, das nur einen echten grünen Test akzeptiert, dann wird der Loop diszipliniert. Review wird Verifikation gegen eine gesetzte Invariante statt Diff-Lektüre. Drei lauffähige, CI-grüne Artefakte belegen das: der Existenzbeweis `runenruf` (46/46), das Gegenstück `ledger-casestudy` mit gefangenem Mis-Spec (5/5, „Falsifiable, after 4 tests“), und drei gestaffelte Verifikationsschichten — Typen, FsCheck-Property, sorry-freier Lean-4-Beweis — auf dem Werterhaltungs-Invarianten.

Der Rest bleibt, und wir verkleinern ihn nicht: das *Setzen* der Invariante. Es ist im System nicht selbst-fundierbar (ein Regress-Resultat, kein Unmöglichkeitbeweis) — Rice (1953) trifft Mensch und Maschine symmetrisch, der Penrose/Lucas-Fehlschluss bleibt abgelehnt. Verifizierer-akzeptiert ist nicht korrekt (VeriAct); deshalb verschiebt sich der Review von Code-gegen-Welt auf Spec-gegen-Welt, er verschwindet nicht. Unser Beitrag ist nicht, diesen Rest zu eliminieren, sondern ihn zu *typisieren*: ihn an genau die Stelle zu legen, an der jemand ihn setzen muss — und wir *wählen* den Menschen, weil Intent extern und verantwortungsbehaftet ist; sichtbar, versioniert, gegen Drift gegatet, sogar im Selbst-Modell des Werkzeugs. Lucke hat die Diagnose; wir liefern die Schraube danach.

9. Reproduzierbarkeit

Alle drei Repositories sind öffentlich auf GitHub und CI-grün. Die folgenden Befehle klonen, bauen und prüfen die jeweilige Kette von Grund auf. Voraussetzungen: .NET SDK 9 für alle drei; zusätzlich

elan (stellt die in lean-toolchain gepinnte Lean-4-Version bereit) für den Werterhaltungs-Beweis in ledger-casestudy.

Koschnag/cong-driven-development (v0.4.0) — die Methode/IDE, Selbst-Modell 66 Knoten / 62 Aligned / 4 Pending, 36/36 Unit-Tests:

```
git clone https://github.com/Koschnag/cong-driven-development
cd cong-driven-development
dotnet test tests/Cdd.Tests    # erwartet: 36/36 grün
```

Koschnag/runenruf — der Existenzbeweis, 46/46 Tests inkl. FsCheck-Property spec-siegel-lager-nichtnegat

```
git clone https://github.com/Koschnag/runenruf
cd runenruf
dotnet test tests/Runenruf.Tests    # erwartet: 46/46 grün
```

Koschnag/ledger-casestudy — das Gegenstück, 5/5 Tests, echtes IO, gefangener Mis-Spec, sorry-freier Lean-Beweis:

```
git clone https://github.com/Koschnag/ledger-casestudy
cd ledger-casestudy
dotnet test tests/Ledger.Tests    # erwartet: 5/5 grün
```

```
# Das Gate beim Scheitern reproduzieren (korrekt grün -> naiv injiziert -> rot -> revert):
./demo-gate-at-failure.sh
```

```
# Den allquantifizierten Lean-4-Beweis prüfen (sorry-frei, ohne Mathlib, Core + omega):
lean proofs/Werterhaltung.lean    # CI-Job: lean-proof, grün auf GitHub-Hardware
# Ausgabe: 'Ledger.werterhaltung' depends on axioms: [propext, Quot.sound] - kein sorryAx
```

10. Literatur

1. Subbarao Kambhampati u. a.: „LLMs Can’t Plan, But Can Help Planning in LLM-Modulo Frameworks”. arXiv:2402.01817, 2024.
2. Shuvendu K. Lahiri: „Intent Formalization: A Grand Challenge for Reliable Coding in the Age of AI Agents”. arXiv:2603.17150, 2026.
3. Md Rakib Hossain Misu, Iris Ma, Cristina V. Lopes: „VeriAct: Beyond Verifiability — Agentic Synthesis of Correct and Complete Formal Specifications”. arXiv:2604.00280, 2026.
4. Margaret-Anne Storey: „From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI”. arXiv:2603.22106, 2026.
5. METR: „Measuring AI Ability to Complete Long Software Tasks”. arXiv:2503.14499, 2025.
6. „Effective LLM Code Refinement via Property-Oriented and Structurally Minimal Feedback” (PGS). arXiv:2506.18315, 2025.
7. Henry Gordon Rice: „Classes of Recursively Enumerable Sets and Their Decision Problems”. Transactions of the American Mathematical Society, Band 74, Nr. 2, Seiten 358–366, 1953.
8. Boris Cherny (Anthropic, Claude Code): zitiert in „Loop Engineering”. The New Stack, 2026. — „my job is to write loops”.

9. Amitayush Thakur u. a.: „CLEVER: A Curated Benchmark for Formally Verified Code Generation”. arXiv:2505.13938, 2025.
10. Lesly Miculicich u. a.: „VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation”. arXiv:2510.05156, 2025.
11. Tessl: „Spec-Driven Development with Tessl” (Framework + Spec-Registry). Agent-Enablement-Plattform, <https://tessl.io>, 2026.
12. Carsten Lucke (TH Mittelhessen): Beitrag zu „Loop Engineering” und Cognitive Debt, LinkedIn, 2026. — die hier konstruktiv adressierte Diagnose; dieses Whitepaper ist die Antwort darauf.